SpaceSec Shield

Phase 2 — Communication Core Development

Lab Build & Handover Documentation

Task ID	SSS-P2
Phase	2 of 5 — Communication Core (Core Networking Layer)
Priority	High (Blocking — Security Layer depends on it)
Status	✓ **DONE**
Dependencies	SSS-P1 (Environment Setup) — complete
Prepared by	Mohammad Hasan Matar
Date	____ / ____ / _____

Purpose of this document: provide a complete, self-contained handover of the Communication Core so that any engineer taking over this lab can understand the two-way command/response design, reproduce the build, replay the space-link simulation, and continue with the Security Layer (Phase 3) without friction.

1. Overview

Phase 2 builds the core communication infrastructure between the Ground Station and the Satellite Node. The single direction baseline from Phase 1 is replaced by a structured, two-way command/response channel over UDP sockets in C++. To make the link realistic, space-communication constraints (latency, jitter, packet loss) are injected on the Satellite Node interface using Linux Traffic Control (tc / netem). The channel is still intentionally unencrypted at this stage; the Security Layer is added in Phase 3.

1.1 Change from Phase 1

Aspect	Phase 1 (baseline)	Phase 2 (this phase)
Direction	One-way (send only)	Two-way (command -> response)
Ground Station	Sends a message, exits	Sends command, waits for ACK
Satellite Node	Receives once, exits	Loops: receive, process, reply
Stability	Not handled	Receive timeout (2 s) prevents hang
Network realism	Clean local link	Latency + loss + jitter via tc/netem

1.2 Node Summary (unchanged from Phase 1)

Node	Role (Phase 2)	OS	Static IP
Ground Station	UDP Sender (command)	Ubuntu 22.04 LTS	192.168.10

Node	Role (Phase 2)	OS	Static IP
		(Desktop)	.10
Satellite Node	UDP Receiver (responder)	Ubuntu Server 22.04 (no GUI)	192.168.10 .20
Attacker (SpaceSec_Kali)	Attack sim (Phase 4)	Kali Linux	192.168.10 .30

Active interface on the Satellite Node: `ens33` (confirmed via `ip a`). All tc commands target this interface, **not eth0**.

2. Core Technology Stack

Component	Choice
Programming Language	C++
Communication Protocol	UDP (User Datagram Protocol)
Networking Layer	BSD Sockets API
Network Simulation	Linux Traffic Control (tc / netem)

✓ g++ and net-tools available on both nodes (from Phase 1)
✓ tc available on the Satellite Node (tc -V -> iproute2-5.15.0)

3. Two-Way UDP Communication

The Ground Station sends a command and blocks for a response with a 2-second receive timeout. The Satellite Node listens in a loop: it receives a command, prints it, builds an acknowledgement, and sends it back to the sender's address. The port used for the channel is 9000.

3.1 Receive timeout (why it matters)

UDP is connectionless — it sends and forgets, with no built-in delivery guarantee. If a command or its reply is lost on the (simulated) space link, a naive receiver would block forever. The Ground Station therefore sets `SO_RCVTIMEO` to 2 seconds via `setsockopt`. On timeout, `recvfrom` returns a negative value and the program reports a graceful “No response” message instead of hanging.

3.2 End-to-end flow

```
Ground Station
  | (1) UDP Command Packet  -->
  v
Satellite Node
  | (2) Process command
  | (3) Build response (ACK: ...)
  |  <-- (4) UDP Response Packet
  v
```

```
Ground Station  (receives ACK, or times out gracefully)
```

4. Build & Run

⚠️ **Note:** Always start the Satellite Node (receiver) FIRST, then run the Ground Station (sender). UDP does not buffer for an absent listener.

4.1 On Satellite Node

```
g++ sat_receiver.cpp -o sat_receiver
./sat_receiver
```

4.2 On Ground Station

```
g++ ground_sender.cpp -o ground_sender
./ground_sender
```

4.3 Expected result (normal conditions)

Ground Station:

```
Command sent: CMD: STATUS_CHECK
Response received: ACK: CMD: STATUS_CHECK
```

Satellite Node:

```
Satellite Node listening on port 9000...
Received Command: CMD: STATUS_CHECK
```

[Paste screenshot here]

Figure 1 — Command sent (Ground Station) and command received + ACK returned (Satellite Node)

5. Space Environment Simulation (tc / netem)

Realistic, unstable space-link conditions are reproduced on the Satellite Node interface (ens33).

⚠️ **One root qdisc only.** You cannot run two “tc qdisc add ... root” commands — the second fails with “File exists”. Add once, then use change to adjust and del to remove.

5.1 Apply combined conditions (one command)

```
sudo tc qdisc add dev ens33 root netem delay 500ms 100ms distribution normal loss 10%
```

Applies 500 ms latency, +/- 100 ms jitter (normal distribution), and 10% packet loss together.

5.2 Adjust conditions afterwards

```
sudo tc qdisc change dev ens33 root netem delay 800ms loss 50%
```

Used to stress the link harder (heavier loss) to demonstrate timeout handling.

5.3 Inspect current settings

```
tc qdisc show dev ens33
# qdisc netem 8001: root refcnt 2 limit 1000 delay 500ms 100ms loss 10%
```

5.4 Remove simulation (reset to normal)

```
sudo tc qdisc del dev ens33 root
tc qdisc show dev ens33
# qdisc fq_codel 0: root ...   (netem removed, link back to normal)
```

[Paste screenshot here]

Figure 2 — Full tc lifecycle on ens33: add -> show -> change -> show -> del -> show

6. Communication Flow Validation

6.1 Normal conditions

With no simulation (or light 10% loss / 500 ms delay), every command is acknowledged. The Ground Station prints “Response received: ACK: CMD: STATUS_CHECK” and the Satellite Node prints “Received Command: CMD: STATUS_CHECK” for each request.

6.2 Degraded conditions (stability proof)

Under heavy degradation (delay 800ms loss 50%), repeated sends produce a mix of outcomes — some succeed with delay, others are lost. Crucially, when a packet is lost the Ground Station does **not** hang; it prints a graceful timeout message and returns to the prompt:

```
Command sent: CMD: STATUS_CHECK
No response (timeout / packet lost)
Command sent: CMD: STATUS_CHECK
Response received: ACK: CMD: STATUS_CHECK
```

[Paste screenshot here]

Figure 3 — Stable behavior under degraded link: mix of successful ACKs and graceful timeouts, no crash/hang

7. Success Criteria & Deliverables

7.1 Definition of Done

- ✓ Commands transmitted successfully over UDP
- ✓ Satellite Node processes commands and returns correct responses (ACK)
- ✓ Network conditions (latency, loss, jitter) applied correctly via tc
- ✓ System remains stable under degraded conditions (timeout handled, no hang/crash)
- ✓ Simulation can be applied and removed cleanly

7.2 Deliverables

- ✓ ground_sender.cpp and sat_receiver.cpp source files (Appendix A)
- ✓ Documented tc command set: add / change / show / del (Section 5)
- ✓ Test evidence: command-response under normal conditions (Figure 1)
- ✓ Test evidence: stable behavior under degraded conditions / timeout (Figure 3)

8. Notes for the Next Engineer

- The interface name is ens33 on the Satellite Node — always replace eth0 in any tc example with it.
- Only one root qdisc may exist. If “File exists” appears, the qdisc is already present — use change or del first.
- Always remove the simulation (tc qdisc del dev ens33 root) before measuring clean connectivity or handing over.
- On the headless Satellite Node, copy/paste from host may not work — type files manually or use a here-doc (cat > file << 'EOF').
- **Out of scope for Phase 2 (coming next):** encryption + secure handshake + packet protection (Phase 3), attack simulation (Phase 4), monitoring dashboard (Phase 5).

Appendix A — Source Code

A.1 sat_receiver.cpp (Satellite Node)

```
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>

int main() {
  int sock = socket(AF_INET, SOCK_DGRAM, 0);
  if (sock < 0) { perror("socket failed"); return 1; }

  sockaddr_in satAddr{};
  satAddr.sin_family = AF_INET;
  satAddr.sin_port   = htons(9000);
  satAddr.sin_addr.s_addr = INADDR_ANY;

  if (bind(sock, (struct sockaddr*)&satAddr, sizeof(satAddr)) < 0) {
    perror("bind failed");
    return 1;
  }

  std::cout << "Satellite Node listening on port 9000..." << std::endl;

  while (true) {
    char buffer[1024] = {0};
    sockaddr_in groundAddr{};
    socklen_t addrLen = sizeof(groundAddr);

    ssize_t n = recvfrom(sock, buffer, sizeof(buffer) - 1, 0,
```

```
                    (struct sockaddr*)&groundAddr, &addrLen);
    if (n < 0) { perror("recvfrom failed"); continue; }
    buffer[n] = '\0';

    std::cout << "Received Command: " << buffer << std::endl;

    // --- Process the command (simulation) ---
    std::string response = "ACK: ";
    response += buffer;

    // --- Send response back to Ground Station ---
    sendto(sock, response.c_str(), response.size(), 0,
           (struct sockaddr*)&groundAddr, addrLen);
  }

  close(sock);
  return 0;
}
```

A.2 ground_sender.cpp (Ground Station)

```
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/time.h>

int main() {
  int sock = socket(AF_INET, SOCK_DGRAM, 0);
  if (sock < 0) { perror("socket failed"); return 1; }

  // --- Receive timeout: stay stable if a reply is lost ---
  timeval tv{};
  tv.tv_sec  = 2;   // wait max 2 seconds for a response
  tv.tv_usec = 0;
  setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

  sockaddr_in satAddr{};
  satAddr.sin_family = AF_INET;
  satAddr.sin_port   = htons(9000);
  satAddr.sin_addr.s_addr = inet_addr("192.168.10.20");

  std::string command = "CMD: STATUS_CHECK";
  ssize_t sent = sendto(sock, command.c_str(), command.size(), 0,
```

```
                    (struct sockaddr*)&satAddr, sizeof(satAddr));
  if (sent < 0) { perror("sendto failed"); return 1; }
  std::cout << "Command sent: " << command << std::endl;

  // --- Wait for response ---
  char buffer[1024] = {0};
  sockaddr_in fromAddr{};
  socklen_t addrLen = sizeof(fromAddr);

  ssize_t n = recvfrom(sock, buffer, sizeof(buffer) - 1, 0,
                    (struct sockaddr*)&fromAddr, &addrLen);
  if (n < 0) {
     std::cout << "No response (timeout / packet lost)" << std::endl;
  } else {
     buffer[n] = '\0';
     std::cout << "Response received: " << buffer << std::endl;
  }

  close(sock);
  return 0;
}
```